

Socket Programming in Java

Complete Manual

Table of Contents

1. Introduction

1.1 What is Socket Programming

1.2 Client-Server Architecture

1.3 TCP vs UDP Sockets

1.4 Use Cases

2. Java Networking Fundamentals

2.1 Java.net Package Overview

2.2 InetAddress Class

2.3 Socket and ServerSocket Classes

2.4 DatagramSocket and DatagramPacket Classes

3. TCP Socket Programming

3.1 TCP Client

3.2 TCP Server

3.3 Full Example: Client-Server Chat

3.4 Step-by-Step Execution Flow

3.5 Exception Handling in TCP

4. UDP Socket Programming

4.1 UDP Client

4.2 UDP Server

4.3 Full Example: File Transfer via UDP

4.4 Limitations and Advantages of UDP

5. Multithreaded Socket Programming

5.1 Why Multithreading is Needed

5.2 Threaded TCP Server Example

5.3 Handling Multiple Clients Concurrently

6. Advanced Topics

6.1 Object Serialization over Sockets

6.2 Using Buffered Streams for Efficient Data Transfer

6.3 Timeouts and Socket Options

6.4 Secure Sockets (SSL/TLS) Overview

7. Debugging and Testing Socket Programs

7.1 Common Errors and Fixes

7.2 Tools for Monitoring Connections

8. Practical Projects Examples

8.1 Chat Application

8.2 Simple Multiplayer Game

8.3 File Sharing System

9. References

1. Introduction

1.1 What is Socket Programming

Socket programming is a way to enable communication between two machines over a network. Java provides the `java.net` package for building socket-based communication programs.

1.2 Client-Server Architecture

- Client: Initiates requests to the server.
- Server: Listens to client requests and responds.

1.3 TCP vs UDP Sockets

Feature	TCP	UDP	Use Case
Connection	Connection-oriented	Connectionless	File transfer, Chat / Video, Gaming
Reliability	Reliable	Not reliable	
Speed	Slower	Faster	

1.4 Use Cases

1. Chat applications
2. File transfer systems
3. Multiplayer games
4. Video streaming and IoT communication

2. Java Networking Fundamentals

2.1 Java.net Package Overview

- `Socket` – TCP client socket
- `ServerSocket` – TCP server socket
- `DatagramSocket` – UDP socket
- `DatagramPacket` – UDP packet

2.2 InetAddress Class

Example:

```
``java
import java.net.*;

public class InetExample
{
    public static void main(String[] args) throws UnknownHostException {
        InetAddress ip = InetAddress.getByName("www.google.com");
        System.out.println("IP Address: " + ip.getHostAddress());
        System.out.println("Host Name: " + ip.getHostName());
    }
}
``
```

2.3 Socket and ServerSocket Classes

- `Socket(String host, int port)` – Connects to a server.
- `ServerSocket(int port)` – Listens for incoming connections.

2.4 DatagramSocket and DatagramPacket Classes

- `DatagramSocket` – Sends and receives UDP packets.
- `DatagramPacket` – Represents data packet.

3. TCP Socket Programming

3.1 TCP Server Example

```
```java
import java.io.*;
import java.net.*;

public class TCPServer {
 public static void main(String[] args)
 {
 try {
 ServerSocket server = new ServerSocket(5000);
 System.out.println("Server is waiting for client...");
 Socket socket = server.accept();

 BufferedReader in = new BufferedReader(new
 InputStreamReader(socket.getInputStream()));

 PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

 String message = in.readLine();
 System.out.println("Client says: " + message);
 out.println("Hello Client!");

 socket.close();
 server.close();
 }
 catch (Exception e)
 {
 e.printStackTrace();
 }
 }
}
```
```

3.2 TCP Client Example

```
``java
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args)
    {
        try
        {
            Socket socket = new Socket("localhost", 5000);

            BufferedReader in = new BufferedReader(new
            InputStreamReader(socket.getInputStream()));

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            out.println("Hello Server!");
            String response = in.readLine();
            System.out.println("Server says: " + response);
            socket.close();
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

| Class , Concept | Descriptions |
|------------------------|----------------------------------|
| ServerSocket | Listens for client requests |
| Socket | Represents connection |
| Port Number | Logical communication endpoint |
| InputStream | Receive data |
| OutputStream | Send data |
| Blocking call | Program waits (accept, readLine) |

| Aspect | TCP (Java) | UDP (Java) |
|---------------|-------------------|-------------------|
| Server class | ServerSocket | DatagramSocket |
| Client class | Socket | DatagramSocket |
| Packet class | ✗ Not used | DatagramPacket |

| Feature | TCP (Java) | UDP (Java) |
|-----------------|-------------------------|-----------------------------------|
| Type | Connection-oriented | Connectionless |
| Classes | Socket,
ServerSocket | DatagramSocket,
DatagramPacket |
| Data form | Stream | Packet |
| Reliability | Guaranteed | Not guaranteed |
| Order | Preserved | Not preserved |
| Speed | Slower | Faster |
| ACK | Implicit | None |
| Example methods | accept(), read() | send(), receive() |

TCP SERVER PROGRAM

```
import java.net.*;
import java.io.*;
public class SquareServer
{
    public static void main(String[] args)
    {
        try
        {
            ServerSocket server = new ServerSocket(6000);
            System.out.println("Server waiting for client...");

            Socket socket = server.accept();

            DataInputStream in =
                new DataInputStream(socket.getInputStream());
            DataOutputStream out =
                new DataOutputStream(socket.getOutputStream());

            int number = in.readInt();
            System.out.println("Number received: " + number);

            int result = number * number;

            out.writeInt(result);
            socket.close();
            server.close();
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}
```


TCP CLIENT PROGRAM :

```
import java.net.*;
import java.io.*;

public class SquareClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 6000);
            DataInputStream in =
                new DataInputStream(socket.getInputStream());
            DataOutputStream out =
                new DataOutputStream(socket.getOutputStream());
            out.writeInt(7); // sending number
            int result = in.readInt();
            System.out.println("Square from server: " + result);
            socket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

UDP SERVER PROGRAM

```
import java.net.*;
public class UDPSquareServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(7000);
            byte[] buffer = new byte[1024];
            System.out.println("UDP Server waiting for client...");
            DatagramPacket request =
                new DatagramPacket(buffer, buffer.length);
            socket.receive(request); // receives data
            String data = new String(request.getData()).trim();
            int number = Integer.parseInt(data);
            System.out.println("Number received: " + number);
            int result = number * number;
            String resultStr = String.valueOf(result);
            byte[] responseData = resultStr.getBytes();

            InetAddress clientAddress = request.getAddress();
            int clientPort = request.getPort();
            DatagramPacket response =
                new DatagramPacket(responseData,
                                    responseData.length,
                                    clientAddress,
                                    clientPort);
            socket.send(response); // sends result
            socket.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

UDP CLIENT PROGRAM

```
import java.net.*;
public class UDPSquareClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            String number = "7";
            byte[] data = number.getBytes();
            InetAddress serverAddress =
                InetAddress.getByName("localhost");
            DatagramPacket request =
                new DatagramPacket(data, data.length, serverAddress,
                    7000);
            socket.send(request); // sends number

            byte[] buffer = new byte[1024];
            DatagramPacket response =
                new DatagramPacket(buffer, buffer.length);

            socket.receive(response); // receives square

            String result =
                new String(response.getData()).trim();

            System.out.println("Square from server: " + result);

            socket.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```